

VERIQTA



aws AWS

FOR COMPLETE BEGINNERS



**A PRACTICAL HANDS-ON GUIDE
FROM ZERO TO YOUR FIRST
AWS PROJECT**



**EASY
EXPLANATIONS**



**HANDS-ON
EXAMPLES**



**SECURITY
BASICS**



**REAL-WORLD
USE CASES**



**BUILD WITH
CONFIDENCE**



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

1. WHAT ACTUALLY HAPPENS WHEN YOU USE AWS

WHAT IS CLOUD COMPUTING?

- Cloud computing means using computers, storage, databases, networking and other services over the internet instead of on your own physical servers.

AWS REGIONS AND AVAILABILITY ZONES

- A Region is a physical location in the world.
- Each Region has multiple isolated locations called Availability Zones (AZs).

DATA CENTERS EXPLAINED SIMPLY

- AWS owns and operates large data centers.
- These data centers contain thousands of servers that run your applications and store your data.

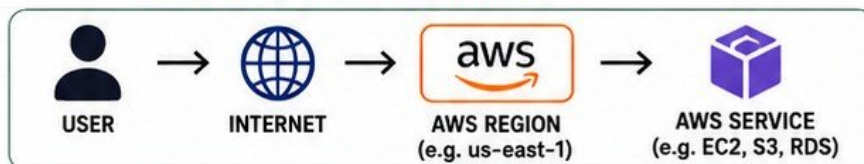
REGION VS AVAILABILITY ZONE

REGION	AVAILABILITY ZONE
- A geographic area (e.g. us-east-1) - Has multiple Availability Zones - Separated by tens to hundreds of km - Independent power, network and cooling	- One or more data centers - Located within a Region - Separated by a few km - Isolated from other AZs in that Region

WHY AWS USES MULTIPLE LOCATIONS

- HIGH AVAILABILITY**
If one location fails, others keep running.
- DISASTER RECOVERY**
Protects your data from large failures.
- LOW LATENCY**
Services are closer to your users.
- SCALABILITY**
AWS can grow without limits.

SIMPLE ARCHITECTURE: HOW IT WORKS



BEGINNER NOTE: WHY CHOOSING A REGION MATTERS

- Affects latency for your users.
- Affects data residency and compliance.
- Affects service availability.
- Affects pricing.

5. CONNECT TO EC2 WITH SSH

WHAT SSH DOES

- SSH (Secure Shell) allows you to connect securely to your EC2 instance and run commands.
- It encrypts your connection so no one can see your data.

PRIVATE KEY PERMISSIONS

- Your private key file must be secure. Set the correct permission so only you can read it.

COMMAND

```
> chmod 400 key.pem
```

This removes access for others and keeps your key secure.

CONNECT TO YOUR EC2 INSTANCE

```
> ssh -i key.pem ubuntu@PUBLIC_IP
```

PART	WHAT IT MEANS
ssh	The command that starts the secure connection.
-i key.pem	Tells SSH which private key to use for authentication.
ubuntu	The default username for the Ubuntu AMI.
PUBLIC_IP	The public IP address of your EC2 instance.

BEGINNER TROUBLESHOOTING FLOW



COMMON SSH ERRORS



PERMISSION DENIED

- Wrong key file permission.
- Wrong username.
- Wrong key file.
- The instance may not be running.

FIX: chmod 400 key.pem
Check username and key file.



CONNECTION TIMEOUT

- Security group does not allow SSH (port 22).
- Instance has no public IP.
- Network or internet issue.

FIX: Check Security Group
Ensure port 22 is allowed.
Check your internet connection.

2. YOUR AWS ACCOUNT AND THE ROOT USER

AWS ACCOUNT STRUCTURE

- An AWS account is your entry point to all AWS services and resources.
- Each account is independent and has its own billing, users, and permissions.
- You control access using IAM (Identity and Access Management).

WHY THE ROOT USER IS DANGEROUS FOR DAILY WORK

- One mistake can affect your entire account.
- Accidental deletion of resources is possible.
- Harder to track actions.
- Target for attackers.
- Credentials are long-term and powerful.

USE THE ROOT USER ONLY WHEN ABSOLUTELY NECESSARY!

AWS MANAGEMENT CONSOLE



- The AWS Management Console is a web interface to access and manage AWS services.
- Sign in using your account email (root user) or IAM user credentials.
- Change the Region in the top-right corner to work in the correct location.

• Console URL: <https://aws.amazon.com/console>

ROOT USER VS IAM USER



ROOT USER

- Created when you open the AWS account.
- Has **unlimited access** to everything.
- Should **NEVER** be used for daily work.
- Secured with email + password + MFA.
- Used only for account setup, billing and critical recovery.

VS



IAM USER

- Created inside AWS by the root user or admin.
- Has **permissions** based on **IAM policies**.
- Used for daily tasks and access to AWS services.
- Can be assigned to groups and roles.
- Best practice: Use IAM users with **least privilege**.

PROTECT YOUR ACCOUNT

1



ENABLE MFA

Enable Multi-Factor Authentication on the root user NOW.

2



SECURE EMAIL

Use a unique, secure email for your AWS account.

3



STRONG PASSWORD

Use a long, strong password that you do not use anywhere else.

4



LIMIT ACCESS

Use IAM users with least privilege for daily tasks.

5



MONITOR ACTIVITY

Use CloudTrail to monitor account activity.

HOW IT ALL CONNECTS



Root user creates and controls access. IAM users and roles do the work.



SECURITY REMINDER

LOCK DOWN THE ROOT ACCOUNT FIRST.
IT IS THE MASTER KEY TO EVERYTHING IN YOUR AWS ACCOUNT.

IAM, USERS, ROLES, AND PERMISSIONS

WHAT IAM CONTROLS

AWS Identity and Access Management (IAM) controls WHO can do WHAT on WHICH AWS resources.



IAM COMPONENTS



USERS

- Represent people
- Have long-term credentials
- Console or CLI access



GROUPS

- Collection of users
- Apply permissions to groups
- Easier management



ROLES

- No long-term credentials
- Assumed by users, services, or AWS resources
- Cross-account access



POLICIES

- JSON documents
- Define permissions (Allow or Deny)
- Attached to users, groups, or roles

ALLOW VS DENY



ALLOW

Grants permission to perform an action.

VS



DENY

Explicit deny always overrides allow.

★ When in doubt, **DENY** is stronger.

LEAST PRIVILEGE



Give only the minimum permissions required to complete a task.

- ✓ Reduce security risks
- ✓ Prevent accidental changes
- ✓ Follow the principle of least privilege

SIMPLE FLOW



USER



IAM POLICY

Defines allowed or denied actions



AWS RESOURCE

EC2, S3, RDS, Lambda, etc.

EXAMPLE

User: Alice

Policy: Allow EC2:StartInstances

Resource: EC2 Instance

Result: Alice can start EC2 instances but cannot delete S3 buckets.



COMMON MISTAKE

Giving AdministratorAccess unnecessarily.

- ✗ Too much access.
- ✗ High security risk.
- ✗ Can lead to accidental or malicious changes.
- ✗ Always use least privilege.



VERIQTAYOUTUBE



VERIQTAGITHUB



VERIQTALINKEDIN



VERIQTAX



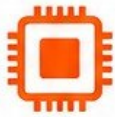
VERIQTAINSTAGRAM



VERIQTATELEGRAM

LAUNCH YOUR FIRST EC2 SERVER

LAUNCHING AN UBUNTU INSTANCE



WHAT IS EC2?

Amazon EC2 (Elastic Compute Cloud) provides resizable virtual servers in the cloud.



INSTANCE TYPES

- Different sizes of CPUs, memory, storage, and network
- Choose the right type for your workload



AMIS

- Amazon Machine Images
- Pre-configured templates (OS, software, settings)
- Example: Ubuntu 22.04 LTS



KEY PAIRS

- Used to securely connect to your instance
- Download the .pem file
- Keep it safe and private



SECURITY GROUPS

- Acts as a virtual firewall
- Control inbound and outbound traffic
- Example: Allow SSH (22), HTTP (80), HTTPS (443)



PUBLIC IP ADDRESSES

- A public IP allows your instance to communicate over the internet
- By default, instances in public subnet get one



LAUNCHING UBUNTU INSTANCE

1. Go to EC2 Dashboard
2. Click Launch Instance
3. Choose Ubuntu 22.04 LTS AMI
4. Select Instance Type (e.g., t2.micro)
5. Configure Key Pair
6. Configure Security Group
7. Add Storage
8. Review and Launch!

ARCHITECTURE: LAPTOP → INTERNET → EC2



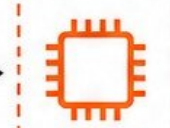
YOUR LAPTOP

You connect to AWS from your device.



INTERNET

Secure connection travels over the internet.



EC2 INSTANCE

Your Ubuntu server running in the AWS cloud.

EXAMPLE CONNECTION



```
ssh -i mykey.pem ubuntu@<PUBLIC_IP>
```

Use your key pair and public IP to connect to your instance.



LEARNING TIP

EC2 IS A VIRTUAL COMPUTER RUNNING INSIDE AWS.
YOU HAVE FULL CONTROL JUST LIKE A PHYSICAL SERVER.



VERIQTAYOUTUBE



VERIQTAGITHUB



VERIQTALINKEDIN



VERIQTAX



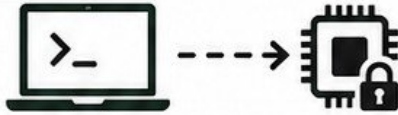
VERIQTAINSTAGRAM



VERIQTATELEGRAM

CONNECT TO EC2 WITH SSH

WHAT SSH DOES



SSH (Secure Shell) lets you connect securely to your EC2 instance and run commands on it.

THE BASIC SSH COMMAND

```
ssh -i key.pem ubuntu@PUBLIC_IP
```

-i key.pem
Tells SSH which private key to use.

ubuntu
The username on the EC2 instance (Ubuntu AMI uses ubuntu by default).

PUBLIC_IP
The public IP address of your EC2 instance.

PRIVATE KEY PERMISSIONS



Your private key file must have strict permissions. Otherwise, SSH will reject it.

SET CORRECT PERMISSIONS

```
chmod 400 key.pem
```

✓ This makes the key readable only by you.

UNDERSTANDING THE COMMAND

- **ssh**
The SSH client (comes built-in on Linux/macOS).
- **-i key.pem**
Uses the specified private key to authenticate.
- **ubuntu**
The default user for Ubuntu AMI. (Use ec2-user for Amazon Linux AMI)
- **PUBLIC_IP**
Example: 54.210.123.45
This is the public IP or DNS of your EC2.

EXAMPLE CONNECTION



```
chmod 400 mykey.pem
```

```
ssh -i mykey.pem ubuntu@54.210.123.45
```



If successful, you will see your Ubuntu welcome message and command prompt.

COMMON SSH ERRORS



PERMISSION DENIED (PUBLICKEY)

CAUSES:

- Wrong key permissions
- Wrong key file
- Wrong username
- Key not added at launch

FIX:

- `chmod 400 key.pem`
- Use the correct key file
- Use the correct username (ubuntu or ec2-user)
- Ensure the instance was launched with this key



CONNECTION TIMEOUT

CAUSES:

- Security group blocking SSH (port 22)
- Wrong public IP or DNS
- Instance is stopped
- Network issues locally

FIX:

- Allow inbound SSH (22) in security group
- Use correct IP or DNS
- Start the instance
- Check your internet connection

BEGINNER TROUBLESHOOTING FLOW



1. CHECK INSTANCE
Is the instance running?



2. CHECK SECURITY GROUP
Is SSH (port 22) allowed from your IP?



3. CHECK KEY FILE
Do you have the correct .pem file?



4. CHECK PERMISSIONS
Run: `chmod 400 key.pem`
Is the permission correct?



5. CHECK USERNAME
Use ubuntu (Ubuntu) or ec2-user (Amazon Linux)



6. TRY SSH AGAIN
`ssh -i key.pem ubuntu@PUBLIC_IP`



STILL NOT WORKING? CHECK YOUR INTERNET, VPC SETTINGS, ROUTE TABLES, AND FIREWALL.



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

EC2 NETWORKING FOR BEGINNERS

PRIVATE IP vs PUBLIC IP



PRIVATE IP (INTERNAL)

- Used inside the VPC
- Not reachable from the internet
- Example: 10.0.1.25



PUBLIC IP (EXTERNAL)

- Reachable from the internet
- Assigned to instances in public subnet
- Example: 54.210.123.45

INBOUND vs OUTBOUND TRAFFIC



INBOUND (INCOMING)

- Traffic coming INTO your EC2 instance
- Controlled by Security Group rules



OUTBOUND (OUTGOING)

- Traffic going OUT from your EC2 instance
- Allowed by default (can be restricted)

SECURITY GROUPS



- Act as a virtual firewall for your EC2 instance
- Control inbound and outbound traffic
- Rule-based (Allow or Deny)
- Applied at the instance level

COMMON PORTS YOU MUST KNOW

22

SSH

Secure Shell Access



Used to securely connect to your EC2 instance

80

HTTP

Web Traffic (Unsecured)



Used for websites without SSL (not secure)

443

HTTPS

Web Traffic (Secured)



Used for secure websites with SSL certificates

TRAFFIC FLOW: HOW IT WORKS



INTERNET

Request comes from the internet



SECURITY GROUP

Checks the rules. If allowed, traffic is permitted.



EC2 INSTANCE

Traffic reaches your EC2 instance.

EXAMPLE FLOW (ALLOW SSH)



Internet



Security Group
Allow 22 (SSH)



EC2 Instance
Port 22 Open

SECURITY GROUP EXAMPLE (INBOUND RULES)

TYPE	PROTOCOL	PORT	SOURCE	DESCRIPTION
SSH	TCP	22	My IP 203.0.113.10/32	Allow SSH from my IP only
HTTP	TCP	80	0.0.0.0/0	Allow web traffic (anywhere)
HTTPS	TCP	443	0.0.0.0/0	Allow secure web traffic (anywhere)
All Traffic	All	All	10.0.0.0/16	Allow internal traffic from VPC only



TIP: Always allow only what is necessary. Restrict sources whenever possible.

COMMON MISTAKE



OPENING EVERY PORT TO THE INTERNET (0.0.0.0/0)

- ✗ Exposes your instance to attacks
- ✗ Increases security risks
- ✗ Violates least privilege principle
- ✗ Only open the ports you need
- ✗ Restrict sources to trusted IPs



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

INSTALL A WEB SERVER ON EC2

DEPLOY NGINX ON YOUR UBUNTU EC2 INSTANCE

STEP-BY-STEP GUIDE

1 UPDATE PACKAGES

```
>_ sudo apt update -y && sudo apt upgrade -y
```

Update the package list and upgrade installed packages.

2 INSTALL NGINX

```
>_ sudo apt install nginx -y
```

Install the Nginx web server.

3 START THE SERVICE

```
>_ sudo systemctl start nginx
```

Start the Nginx service.

4 CHECK SERVICE STATUS

```
>_ sudo systemctl status nginx
```

Make sure Nginx is active and running.

5 ALLOW HTTP TRAFFIC (PORT 80)

```
sudo ufw allow 'Nginx Full'
```

Or allow port 80 in your Security Group.

6 ACCESS FROM BROWSER



Open your browser and visit:

http://YOUR_EC2_PUBLIC_IP

You should see the Nginx welcome page!

WHAT IS HAPPENING?

NGINX

Nginx is a high-performance web server that handles HTTP requests and delivers web content.

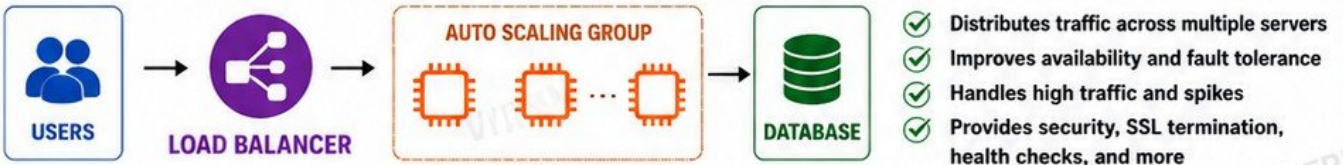
TRAFFIC FLOW: HOW IT WORKS



EXAMPLE OUTPUT



PRODUCTION PREVIEW: WHY APPS NORMALLY SIT BEHIND LOAD BALANCERS



COMMON MISTAKE

Running a web server without security or behind a load balancer.
Single server = Single point of failure.

- ✗ Do not expose unnecessary ports.
- ✗ Do not use public IP for databases.
- ✗ Always follow the principle of least privilege.
- ✗ Monitor your server and keep it updated.



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

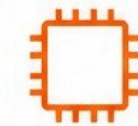
AMAZON EBS, YOUR EC2 DISK

WHAT EBS IS



Amazon EBS (Elastic Block Store) provides persistent block storage for EC2 instances. It acts like a hard drive for your virtual server.

ROOT VOLUMES



EC2 INSTANCE



ROOT VOLUME (EBS)

- Every EC2 instance has a root volume.
- Contains the OS and boot files.

VOLUME TYPES

gp3

General Purpose SSD

- Balanced price and performance
- Default choice for most workloads

io2

Provisioned IOPS SSD

- Highest performance
- For mission-critical applications
- Consistent I/O performance

st1

Throughput Optimized HDD

- Low cost
- High throughput
- For big data, data lakes

sc1

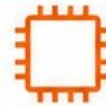
Cold HDD

- Lowest cost
- Infrequently accessed data
- Archives, backups

ATTACH A VOLUME



EBS VOLUME



ATTACH TO EC2 INSTANCE



READY FOR USE

MOUNT A VOLUME (ON LINUX)

- 1 Identify the device (e.g., /dev/xvdf)
- 2 Create a file system (mkfs -t ext4 /dev/xvdf)
- 3 Create a mount point (mkdir /data)
- 4 Mount the volume (mount /dev/xvdf /data)
- 5 Verify with: df -h



EC2 INSTANCE



MOUNT POINT /data

SNAPSHOTS



EBS VOLUME



SNAPSHOT (Point-in-time backup)



S3 BUCKET (Snapshot stored here)

- Use snapshots for backup
- Restore to a new volume
- Supports cross-region copies

EC2 VS EBS



EC2 (COMPUTE)

- Virtual server
- Provides CPU, RAM, and networking
- Can be stopped, terminated, or replaced

VS



EBS (STORAGE)

- Block storage (disk)
- Persistent
- Independent of the EC2 instance
- Survives instance stop and start

WHAT HAPPENS TO STORAGE?



STOP INSTANCE

- EBS volumes remain intact
- You are not charged for EC2, but EBS charges continue



START INSTANCE

- Root volume re-attaches
- Data is still there



TERMINATE INSTANCE

- Root volume is deleted by default
- Any additional EBS volumes remain unless you delete them



BEGINNER NOTE

COMPUTE AND STORAGE ARE SEPARATE CONCEPTS. YOU CAN CHANGE, STOP, OR TERMINATE COMPUTE WITHOUT LOSING YOUR DATA (IF STORED IN EBS).



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

AMAZON S3, OBJECT STORAGE

WHAT IS S3?



Amazon S3 (Simple Storage Service) is highly durable object storage for the cloud.

Store and retrieve any amount of data from anywhere.

KEY CONCEPTS



BUCKETS

Containers for your objects. Bucket names must be unique globally.



OBJECTS

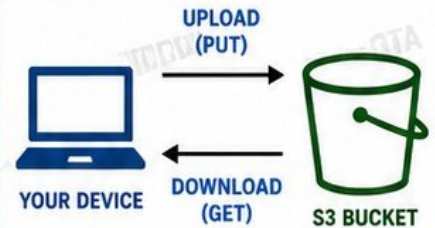
The files you store in S3. Each object consists of data and metadata.



OBJECT KEYS

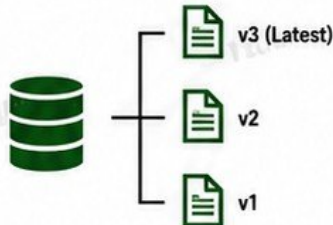
The unique path (name) that identifies an object inside a bucket.
Example: images/cat.jpg

UPLOAD & DOWNLOAD



✓ You can upload or download objects using the AWS Console, CLI, SDKs, or applications.

VERSIONING



Versioning keeps multiple versions of an object so you can restore previous versions if needed.

STORAGE CLASSES



S3 Standard
Frequent access, low latency



S3 Intelligent-Tiering
Automatically moves data to save money



S3 Standard-Infrequent Access (IA)
Lower cost for less frequent access



S3 One Zone-Infrequent Access
Lower cost, stored in one AZ



S3 Glacier Instant Retrieval
Archival, milliseconds retrieval



S3 Glacier Deep Archive
Lowest cost, hours retrieval

BUCKET PERMISSIONS



ACCESS CONTROL

Control who can access your bucket and objects.



BUCKET POLICY

JSON policy that defines permissions for the bucket and objects.



IAM PERMISSIONS

Use IAM users, groups, or roles to manage access.

S3 VS EBS



S3 (OBJECT STORAGE)

- Stores objects (files)
- Access over the internet
- Virtually unlimited scale
- Highly durable (11 9's)
- Pay for storage used
- Best for backups, static files, data lakes, media, archives



EBS (BLOCK STORAGE)

- Stores blocks (disks)
- Attached to EC2 instances
- Limited to AZ
- Highly durable (99.999%)
- Pay for provisioned size
- Best for OS, databases, applications

COMMON USE CASES



Backup & Restore



Static Website Hosting



Data Lakes & Analytics



Media Storage (Images, Videos)



Logs & Archival Data



Application File Storage



SECURITY REMINDER

DO NOT MAKE BUCKETS PUBLIC UNNECESSARILY!

Public access can expose your data to the entire internet. Use least privilege and follow security best practices.



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

BUILD YOUR FIRST VPC MENTAL MODEL

WHAT IS A VPC?



A VPC (Virtual Private Cloud) is your private network in AWS. You control your IP address range, subnets, route tables, and gateways.

CIDR BLOCKS

A CIDR block is the IP address range for your VPC.

10.0.0.0/16

This gives you 65,536 IP addresses. You can create smaller subnets inside this range.

Examples:

- 10.0.0.0/16 (VPC)
- 10.0.1.0/24 (Public Subnet A)
- 10.0.2.0/24 (Private Subnet A)

KEY COMPONENTS



PUBLIC SUBNETS

Have a route to an Internet Gateway. Resources can be internet-facing.



PRIVATE SUBNETS

No direct route to the Internet. Resources are internal only.



ROUTE TABLES

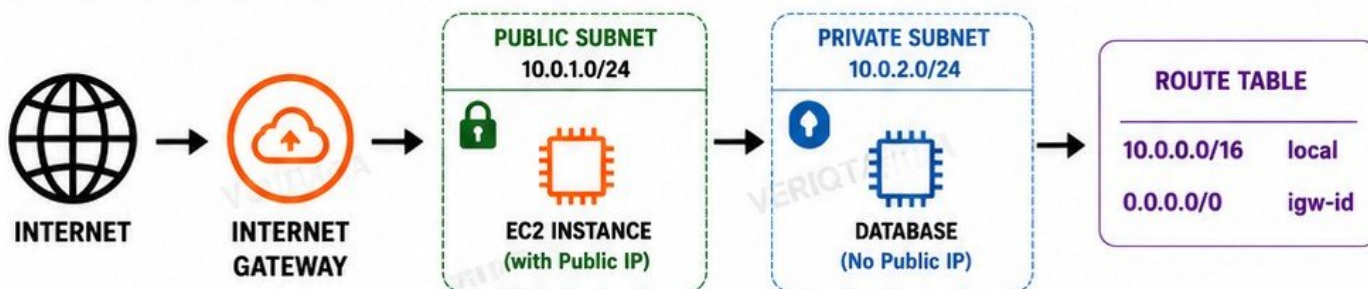
Control where network traffic goes. Each subnet is associated with a route table.



INTERNET GATEWAY (IGW)

Allows resources in public subnets to communicate with the Internet.

SIMPLE ARCHITECTURE



1 Traffic from the Internet comes in.

2 The Internet Gateway allows traffic into the VPC.

3 Public Subnet routes traffic to the EC2 instance.

4 Private Subnet resources (like databases) are not directly accessible from the Internet.

WHY PRIVATE RESOURCES SHOULD NOT BE AUTOMATICALLY INTERNET-FACING

- ✓ Reduces attack surface and security risks.
- ✓ Protects sensitive data and internal systems.
- ✓ Limits exposure to only what is necessary.
- ✓ Use private subnets for databases, caches, internal APIs, and back-end services.
- ✓ Access privately using bastion hosts, VPN, Direct Connect, or Load Balancers.



QUICK RECAP

	VPC = Your private network in AWS
	CIDR = IP address range for your VPC
	Public Subnet = Internet-facing resources
	Private Subnet = Internal-only resources
	Route Table = Controls traffic flow
	Internet Gateway = Connects to the Internet



SECURITY REMINDER

DO NOT MAKE BUCKETS PUBLIC UNNECESSARILY!

Public access can expose your data to the entire internet. Use least privilege and follow security best practices.

PUBLIC AND PRIVATE SUBNETS

✓ WHAT MAKES A SUBNET PUBLIC?

- Route table has a route to the Internet Gateway (IGW)
0.0.0.0/0 → igw-xxxxxxx
- Resources can have a Public IP
- Accessible directly from the internet

🔒 WHAT MAKES A SUBNET PRIVATE?

- No direct route to Internet Gateway
- Route table has default route to NAT Gateway instead
0.0.0.0/0 → nat-xxxxxxxx
- Resources cannot have a Public IP
- Not accessible directly from the internet

🔗 NAT GATEWAY CONCEPT

- Located in a public subnet
- Has a route to IGW
- Private subnets route outbound traffic to NAT Gateway
- Allows private resources to access the internet for updates, APIs, etc.
- Internet cannot start connections to private resources

💻 PUBLIC IP ADDRESSING

- Public IP = Reachable from the internet
- Can be auto-assigned on launch in public subnets
- Private IP = Internal communication inside the VPC
- Never exposes your resource directly

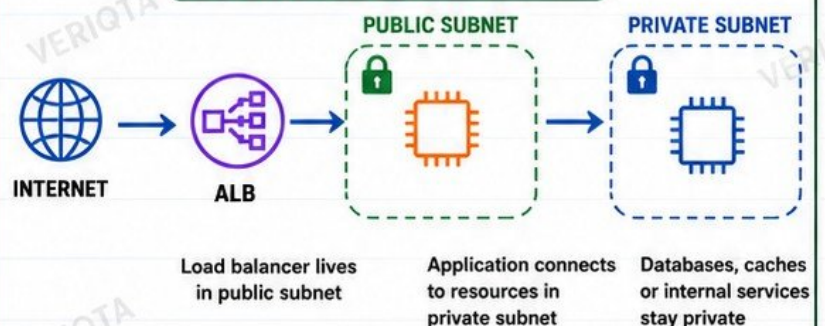
➕ ROUTE TABLES

- Control where network traffic goes
- Each subnet must be associated with a route table
- Routes decide if traffic goes to IGW, NAT Gateway or stays private

🌐 INTERNET GATEWAY (IGW)

- Attaches to your VPC
- Allows communication between your VPC and the internet
- Used only by public subnets through route tables

ARCHITECTURE FLOW



COMMON MISTAKE

THINKING THE SUBNET NAME MAKES IT PUBLIC.

A SUBNET IS PUBLIC OR PRIVATE BASED ON ITS ROUTE TABLE,
NOT ITS NAME.

✗ EXAMPLES

✗ "public-subnet" with no route to IGW → NOT PUBLIC

✗ "private-subnet" with route to IGW → PUBLIC

✓ ALWAYS CHECK THE ROUTE TABLE



KEY TAKEAWAY

- ✓ Public subnet = Route to IGW + Public IPs + Internet access
- ✓ Private subnet = Route to NAT + No Public IPs + No direct internet
- ✓ Route tables decide, not the subnet name
- ✓ Use NAT Gateway for private subnet outbound internet access

REMEMBER

IGW = Inbound + Outbound

NAT = Outbound Only

Route Table = Traffic Rules



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

DNS WITH AMAZON ROUTE 53



1. WHAT DNS DOES

- DNS (Domain Name System) translates human-readable names into IP addresses.
- It acts like the internet's phonebook.
- Without DNS, you would need to remember IP addresses.



2. DOMAIN NAMES & IP ADDRESSES

- Domain name: easy for humans (app.example.com)
- IP address: used by machines (192.0.2.10)
- DNS connects the two.



3. HOSTED ZONES

- A hosted zone is a container for DNS records for a domain.
- Public Hosted Zone: resolves names on the internet.
- Private Hosted Zone: resolves names inside your VPC.
- Example: example.com



5. TTL (TIME TO LIVE)

- TTL determines how long DNS information is cached.
- Lower TTL = faster changes, more DNS queries.
- Higher TTL = fewer DNS queries, longer caching.
- Common values: 30, 60, 300, 900, 3600 seconds.

THE DNS FLOW



USER BROWSER

Types:

app.example.com



AMAZON ROUTE 53

Resolves the name using DNS records



AWS RESOURCE

Returns the IP address of the destination



Flow: app.example.com → Route 53 → AWS Resource



4. DNS RECORDS

DNS records tell Route 53 how to respond to a DNS query.

A RECORD

Maps a domain or subdomain to an IPv4 address.

Example:
app.example.com
→ 192.0.2.10



CNAME RECORD

Maps one domain name to another domain name.

Example:
www.example.com
→ app.example.com



ALIAS RECORD

Maps a domain to an AWS resource (like ALB, CloudFront, S3, etc.).

Example:
app.example.com
→ dualstack.alb.amazonaws.com



COMMON RECORD TYPES SUMMARY

RECORD TYPE	PURPOSE	POINTS TO	EXAMPLE
A	IPv4 address	IPv4 address	app.example.com → 192.0.2.10
CNAME	Another domain	Domain name	www.example.com → app.example.com
ALIAS	AWS resource	AWS resource	app.example.com → dualstack.alb.amazonaws.com
TTL	Cache duration	Time in seconds	TTL = 300



LEARNING TIP

DNS TRANSLATES NAMES INTO DESTINATIONS.
THINK OF DNS AS THE INTERNET'S GPS.



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

ELASTIC LOAD BALANCING



1. WHY ONE EC2 INSTANCE IS RISKY

- Single point of failure
- If the server crashes, your application goes down
- Cannot handle high traffic
- No automatic recovery
- Maintenance requires downtime

2. ARCHITECTURE OVERVIEW



EC2-1



EC2-2

USERS

APPLICATION
LOAD BALANCER

Flow: Users → ALB → EC2-1 / EC2-2



3. APPLICATION LOAD BALANCER (ALB)

- Operates at Layer 7 (HTTP/HTTPS)
- Routes traffic based on content (Path, Host, Headers)
- Supports containers, microservices, and modern applications
- Provides SSL/TLS termination
- Built-in health checks and metrics



4. LISTENERS

- Process connection requests on a specific port (80, 443, etc.)
- Forward requests to target groups
- Can redirect HTTP to HTTPS
- Support multiple rules (path-based, host-based)

Example:

Listener on port 443 → HTTPS traffic to your application



5. TARGET GROUPS

- A collection of targets (EC2 instances, IPs, or containers)
- ALB sends traffic only to healthy targets
- Define the protocol and port for the targets
- Enable easy scaling and deployment

Example:

Target Group: app-tg
Targets: EC2-1, EC2-2



6. HEALTH CHECKS

- ALB periodically checks the health of each target
- Unhealthy targets are removed from traffic
- Ensures only healthy instances receive requests
- Customizable path, interval, thresholds



7. TRAFFIC DISTRIBUTION

- Distributes incoming traffic across healthy targets
- Uses round-robin algorithm (by default)
- Evenly balances load
- Automatically adjusts when targets are unhealthy or removed



8. BENEFITS

- High availability
- Scalability
- Fault tolerance
- Zero or minimal downtime
- Better performance
- Secure and reliable applications



PRODUCTION PREVIEW

LOAD BALANCING REMOVES DEPENDENCE ON ONE SERVER.

- ✓ Keeps your application available
- ✓ Handles traffic spikes
- ✓ Enables rolling updates
- ✓ Improves reliability and user experience

WITHOUT LOAD BALANCER

- ✗ Single server handles all traffic
- ✗ Server crash = application down
- ✗ Cannot scale easily
- ✗ Maintenance causes downtime



EC2-1
(SINGLE INSTANCE)

VS

WITH LOAD BALANCER



ALB



EC2-1



EC2-2

- ✓ Traffic distributed across multiple servers
- ✓ If one server fails, others continue serving
- ✓ Easily scales to handle more traffic
- ✓ Maintenance with no downtime



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

AUTO SCALING



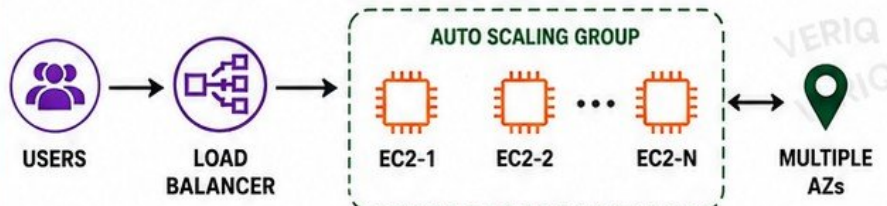
1. WHY APPLICATIONS NEED SCALING

- Traffic changes throughout the day.
- More users = more load.
- Without scaling, users experience slow apps or errors.
- Scaling ensures high availability, performance, and a great user experience.



2. AUTO SCALING GROUPS (ASG)

- A collection of EC2 instances treated as a single unit.
- Automatically adds or removes EC2 instances based on rules.
- Spans multiple Availability Zones for high availability.
- Works with Load Balancers to distribute traffic.



3. CAPACITY SETTINGS IN AN ASG



MINIMUM CAPACITY

- The minimum number of instances that must always be running.
- Protects your app during low traffic.



DESIRED CAPACITY

- The ideal number of instances you want running.
- Auto Scaling tries to keep the group at this number.



MAXIMUM CAPACITY

- The maximum number of instances allowed.
- Prevents uncontrolled scaling and high costs.



4. LAUNCH TEMPLATES

- Defines how instances are launched.
- Includes AMI, instance type, key pair, security groups, user data, etc.
- Ensures every instance is identical and consistent.
- Preferred over Launch Configurations.



5. SCALING POLICIES

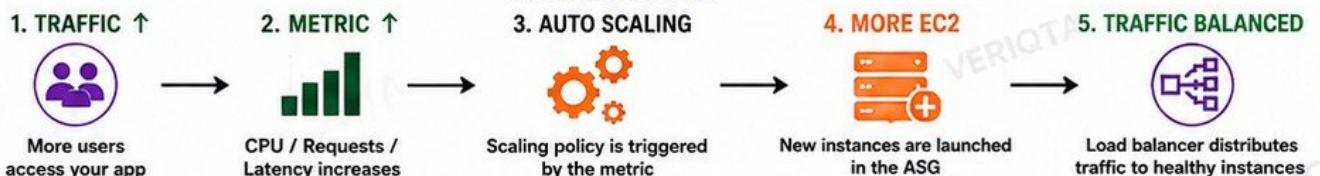
- Define when and how to scale.
- Types:
 - Target Tracking Scaling
 - Step Scaling
 - Simple Scaling
 - Scheduled Scaling



6. FAILED INSTANCE REPLACEMENT

- ASG continuously monitors instance health.
- Unhealthy instances are automatically terminated.
- New healthy instances are launched to maintain Desired Capacity.
- Ensures high availability and reliability.

7. SCALING FLOW



Flow: Traffic ↑ → Metric ↑ → Auto Scaling → More EC2



KEY BENEFITS



HIGH AVAILABILITY



SCALABILITY ON DEMAND



COST OPTIMIZATION



BETTER PERFORMANCE



SELF-HEALING ARCHITECTURE



REMEMBER

- Set the right Min, Desired, and Max values.
- Use the right Scaling Policy.
- Always monitor your metrics.
- Test scaling in real scenarios.



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

AMAZON RDS

MANAGED DATABASES



1. WHAT IS RDS?

Amazon Relational Database Service (RDS) is a fully managed service that makes it easy to set up, operate, and scale relational databases in the cloud.

- Handles provisioning, patching, backups, monitoring, and more.
- You focus on your application, not database administration.



2. DATABASE ENGINES

- Choose from popular engines:



MySQL



PostgreSQL



MariaDB



Oracle



Microsoft SQL Server



Amazon Aurora
(MySQL / PostgreSQL)



3. DATABASE INSTANCES

- An RDS instance is the running database.
- You can choose instance type (size) based on your workload.
- Supports scaling up or down with minimal downtime.

Example:
db.t3.medium
(2 vCPU, 4 GiB RAM)



4. STORAGE

- RDS uses EBS for storage.
- Automatically scales as your data grows.
- Choose storage type:
 - ✓ General Purpose (SSD)
 - ✓ Provisioned IOPS (SSD)
 - ✓ Magnetic (HDD)
- You do NOT manage disks.



5. BACKUPS

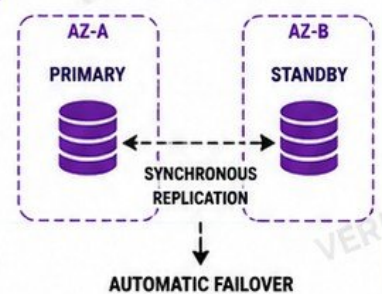
- Automated daily backups.
- Point-in-time recovery (PITR) to any second.
- Backup retention configurable (1–35 days).
- Snapshots for long-term storage or migration.



6. MULTI-AZ CONCEPT

- Creates a standby replica in a different Availability Zone.
- Automatic failover if the primary goes down.
- Improves availability and durability.
- Recommended for production workloads.

MULTI-AZ DEPLOYMENT

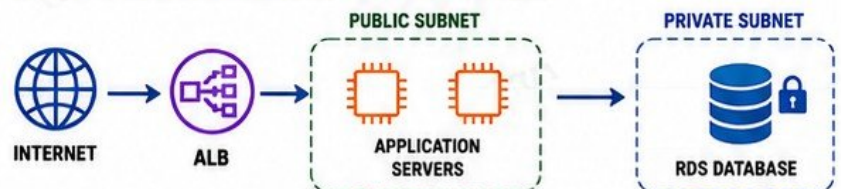


7. SECURITY GROUPS

- Acts as a virtual firewall for your DB.
- Control which IPs or resources can connect.
- Only allow access from your application servers.
- Do NOT open to the internet (0.0.0.0/0) for production!

Example: Allow only EC2 SG (app-sg) to connect to RDS

8. WHY DATABASES BELONG IN PRIVATE SUBNETS



- ✓ Databases store sensitive data.
- ✓ Private subnets are not directly accessible from the internet.
- ✓ Only application tier can communicate with the database.
- ✓ This architecture increases security and protects your data.

9. ARCHITECTURE: APPLICATION → RDS



10. COMMON MISTAKE

EXPOSING A PRODUCTION DATABASE PUBLICLY!

- ✗ Opening RDS to the internet (0.0.0.0/0) is a major security risk.
- ✗ Attackers can scan and attempt to access your database.
- ✗ Always restrict access to trusted sources only.
- ✗ Keep RDS in private subnets and use security groups.



LEARNING TIP:

RDS handles the heavy lifting so you can build, scale, and secure your applications with a reliable database.

- ✓ Easy to deploy
- ✓ Highly available
- ✓ Backups & recovery built-in

- ✓ Scalable
- ✓ Secure
- ✓ Cost-effective



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

MONITORING WITH AMAZON CLOUDWATCH

SEE EVERYTHING. UNDERSTAND EVERYTHING. ACT BEFORE USERS NOTICE.



METRICS

Numeric data about your resources over time.



LOGS

Detailed logs from your applications and services.



ALARMS

Alerts you when metrics cross thresholds you define.



DASHBOARDS

Visualize and monitor multiple metrics in one place.



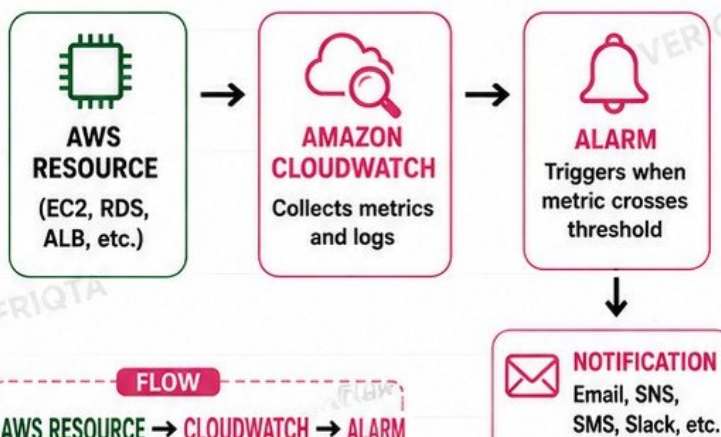
NOTIFICATIONS

Send alerts via Email, SNS, SMS or other integrations.

EC2 METRICS YOU SHOULD WATCH

METRIC	WHAT IT MEANS
CPU UTILIZATION	How much CPU your instance is using.
STATUS CHECK	Health of the instance and underlying hardware.
NETWORK IN/OUT	Amount of network traffic in and out.
DISK READ/WRITE	Disk input/output activity.
DISK SPACE USED	How much storage is used on your instance.

HOW ALARMS WORK



ALARM THRESHOLDS

- Define the metric
- Set the threshold value
- Choose the period
- Set evaluation periods
- Choose the statistic
- Choose the alarm action

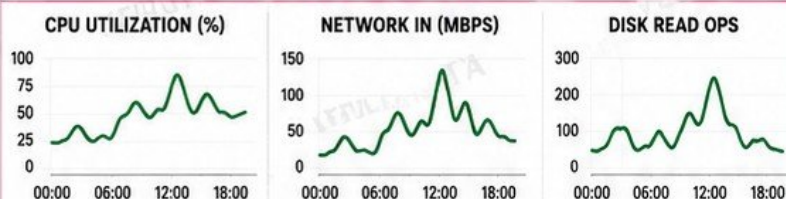
EXAMPLE: CPU UTILIZATION ALARM

- Metric: CPUUtilization
- Threshold: Greater than 80%
- Period: 5 Minutes
- Evaluation: 2 out of 3
- Action: Send email + SNS

CLOUDWATCH LOGS

- Collect logs from EC2, Lambda, Containers, Applications and more.
- Centralize logs in Log Groups.
- Search and analyze with CloudWatch Logs Insights.
- Set log retention to control costs.
- Use logs for troubleshooting and auditing.

DASHBOARD EXAMPLE



SEE TRENDS. SPOT ISSUES. TAKE ACTION.



BEGINNER NOTE

RUNNING INFRASTRUCTURE WITHOUT MONITORING MEANS OPERATING BLIND.

DON'T FLY BLIND.

MONITOR EVERYTHING THAT MATTERS.



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

AWS CLI FOR BEGINNERS

THE FAST, POWERFUL WAY TO MANAGE AWS.

WHY ENGINEERS USE THE CLI



- ✓ Faster than clicking through the console
- ✓ Automate repetitive tasks
- ✓ Script deployments and operations
- ✓ Work from anywhere (terminal, CI/CD, servers)
- ✓ Essential skill for DevOps and Cloud Engineers

CLI WORKS EVERYWHERE



LAPTOP



SERVERS



CI/CD



CLOUD SHELL

ONE TOOL. ENDLESS POSSIBILITIES.

1. INSTALL AWS CLI

WINDOWS

Download and run
AWS CLI MSI Installer
from:
<https://aws.amazon.com/cli/>

Verify:

```
aws --version
```

macOS

Run in terminal:

```
brew install awscli
```

Verify:

```
aws --version
```

LINUX

Run in terminal:

```
sudo apt update  
sudo apt install awscli
```

Verify:

```
aws --version
```

CLOUDSHELL

AWS CLI comes
pre-installed.
Just start using it!

2. CONFIGURE AWS CLI

Run the command:

```
aws configure
```

You will be prompted for:

- ⊗ AWS Access Key ID Your access key
- ⊗ AWS Secret Access Key Your secret key
- 🌐 Default region name Example: us-east-1
- 📄 Default output format Example: json

3. PROFILES (MULTIPLE ACCOUNTS)

Use different profiles for different accounts
or environments.

Configure a profile:

```
aws configure --profile myprofile
```

Use a profile:

```
aws s3 ls --profile myprofile
```

View your active profile:

```
aws configure list --profile myprofile
```

4. REGIONS

List all available regions:

```
aws ec2 describe-regions
```

Set default region (example):

```
aws configure set region us-east-1
```

5. CREDENTIALS

CLI uses credentials from:

- ✓ aws configure (stored in ~/.aws/credentials)
- ✓ Environment variables
- ✓ IAM Roles (EC2, ECS, Lambda)
- ✓ AWS SSO (IAM Identity Center)

6. COMMON AWS CLI COMMANDS FOR BEGINNERS

WHO AM I?

```
aws sts get-caller-identity
```

Shows your AWS account details
and the user/role you are using.



LIST S3 BUCKETS

```
aws s3 ls
```

Lists all S3 buckets you have
access to.



LIST EC2 INSTANCES

```
aws ec2 describe-instances
```

Shows all EC2 instances in
your account.



SECURITY REMINDER

NEVER COMMIT AWS CREDENTIALS TO GIT.

Protect your keys. Protect your account. Protect your career.

NEVER STORE IN CODE:

- ⊗ Access Keys
- ⊗ Secret Keys
- ⊗ Credentials Files



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

UNDERSTAND AWS PRICING

BEFORE YOU GET A SURPRISE BILL

1. PAY-AS-YOU-GO



- ✓ You pay only for what you use.
- ✓ No upfront commitment.
- ✓ Billed by the second, hour, day, or month depending on the service.
- ✓ Stop or delete resources to stop the charges.

3. FREE TIER LIMITATIONS



- Free Tier is for 12 months from account creation.
- Limited to specific services and usage amounts.
- Not all services are free.
- Exceeding limits = normal charges.
- Always check the current Free Tier details.

2. COMMON AWS COST COMPONENTS



EC2 COSTS

Charged by instance type, region, OS, running time, and additional features like EBS, IPs, and snapshots.



STORAGE COSTS

EBS, S3, RDS storage, backups, and snapshots all incur charges.



DATA TRANSFER

Data out to the internet is charged. Data between AWS services in the same region is usually free.



NAT GATEWAY COSTS

Charged per hour + per GB of data processed. Can get expensive!

4. AWS BUDGETS & COST ALERTS



AWS BUDGETS

- ✓ Set a monthly budget for your account.
- ✓ Track actual vs budgeted spend.
- ✓ Get notified before you exceed limits.
- ✓ Budgets are FREE!



COST ALERTS

- ✓ Receive alerts when costs reach a certain threshold.
- ✓ Alerts via Email or SNS (SMS, Slack, etc.)
- ✓ Helps you take action early and avoid unexpected bills.

5. STOPPING vs TERMINATING RESOURCES



STOPPING (EC2 EXAMPLE)

- ✓ Stops the instance (it's not running).
- ✓ You stop paying for Compute (EC2) charges.
- ✓ You still pay for attached EBS volumes, Elastic IPs, and some other resources.
- ✓ You can start it again anytime.

VS



TERMINATING (EC2 EXAMPLE)

- ✓ Deletes the instance completely.
- ✓ You stop paying for Compute and most associated resources.
- ✓ EBS volumes are deleted (if not saved).
- ✓ You cannot start it again.

6. COST CONTROL BEST PRACTICES



- ✓ Use the right-sized instance.
- ✓ Stop or terminate idle resources.
- ✓ Use Auto Scaling to match demand.
- ✓ Use S3 Intelligent-Tiering for storage.
- ✓ Monitor costs with Budgets and Alerts.
- ✓ Review the AWS Cost Explorer regularly.

7. BEGINNER RULE



**DELETE RESOURCES
CREATED ONLY FOR PRACTICE.**
CLOUD IS NOT FREE.
**WHAT YOU DON'T USE,
YOU MUST TURN OFF OR DELETE.**



WATCH TODAY, OR PAY TOMORROW.
MONITOR. OPTIMIZE. SAVE.



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM

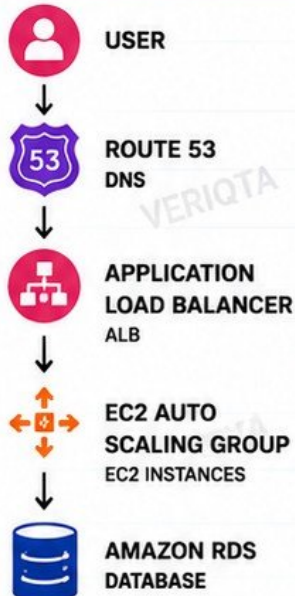


VERIQTA TELEGRAM

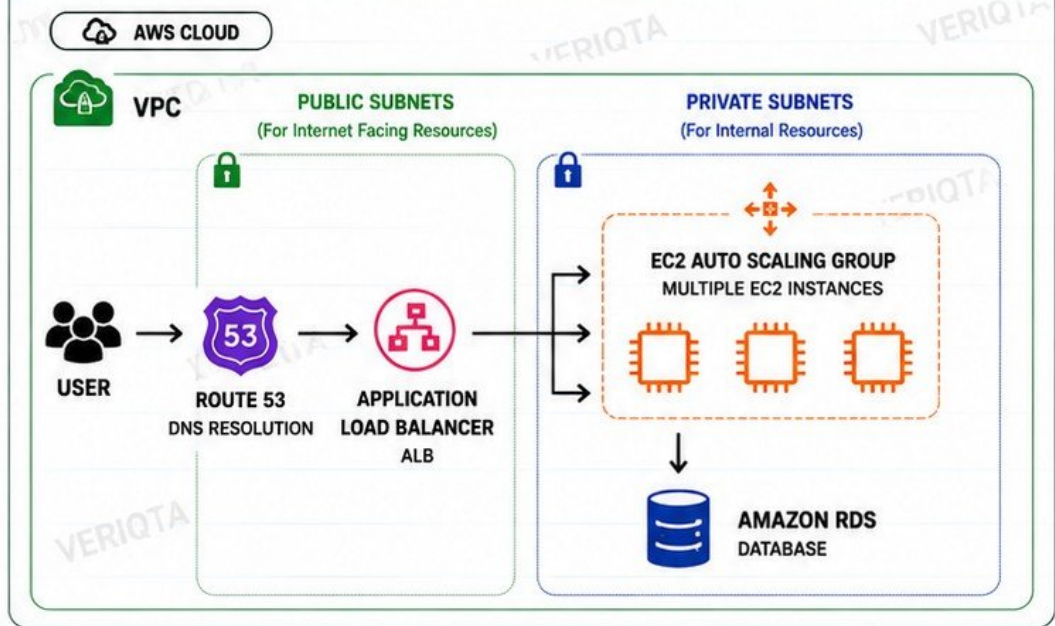
BUILD YOUR FIRST AWS ARCHITECTURE

A SIMPLE, SCALABLE, AND PRODUCTION-READY ARCHITECTURE

BUILD:



ARCHITECTURE DIAGRAM



SUPPORTING SERVICES



HOW A REQUEST TRAVELS THROUGH THE ARCHITECTURE



KEY POINTS TO REMEMBER



- ✓ Route 53 handles DNS, not hosting.
- ✓ ALB handles traffic distribution and health checks.
- ✓ EC2 ASG provides high availability and scalability.
- ✓ RDS is managed and highly available.
- ✓ Keep databases in private subnets.
- ✓ Always use Security Groups to control access.

BEST PRACTICES

- ✓ Use least privilege IAM roles.
- ✓ Enable CloudWatch Alarms for critical metrics.
- ✓ Use S3 for logs, backups, and static content.
- ✓ Back up your RDS database regularly.
- ✓ Test your architecture by simulating failures.

BEGINNER NOTE



THIS ARCHITECTURE IS THE FOUNDATION OF MOST REAL-WORLD AWS APPLICATIONS.



VERIQTA YOUTUBE



VERIQTA GITHUB



VERIQTA LINKEDIN



VERIQTA X



VERIQTA INSTAGRAM



VERIQTA TELEGRAM

FIRST AWS HANDS-ON PROJECT

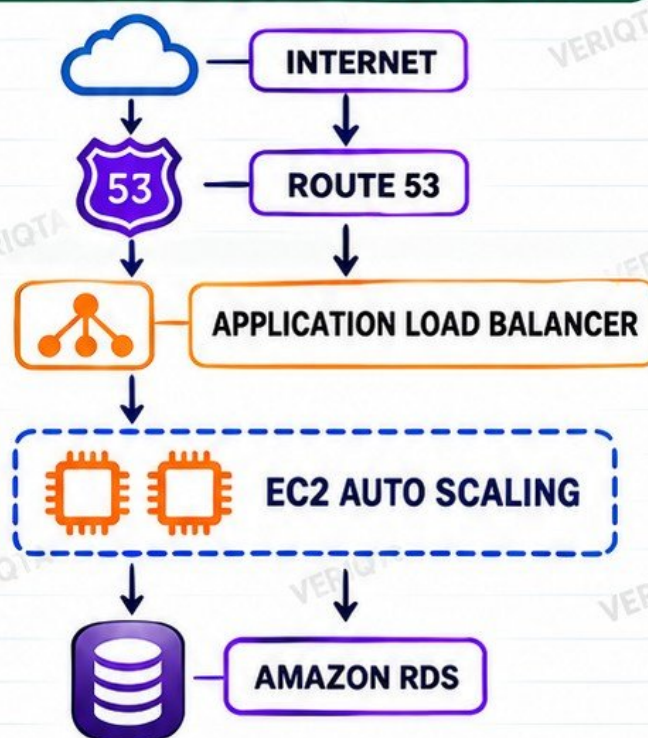
DEPLOY A SMALL WEB APPLICATION



BUILD THE ENVIRONMENT YOURSELF:

- ✓ Create a VPC
- ✓ Create public and private subnets
- ✓ Configure route tables
- ✓ Attach an Internet Gateway
- ✓ Create security groups
- ✓ Launch EC2
- ✓ Connect using SSH
- ✓ Install Nginx
- ✓ Deploy a simple application
- ✓ Create an Application Load Balancer
- ✓ Configure a target group
- ✓ Add Auto Scaling
- ✓ Create an RDS database
- ✓ Store static objects in S3
- ✓ Configure Route 53
- ✓ Add CloudWatch monitoring
- ✓ Create a billing alert
- ✓ Test the application
- ✓ Break one component deliberately
- ✓ Troubleshoot the failure
- ✓ Clean up all resources

FINAL ARCHITECTURE



**S3 → STATIC/
OBJECT STORAGE**



**CLOUDWATCH →
MONITORING**



**IAM →
ACCESS CONTROL**



BUILD
Environment &
Resources



TEST
Application
Works



BREAK
One Component
Deliberately



TROUBLESHOOT
Find and Fix the
Failure



CLEAN UP
All Resources



LEARNING GOAL: Build, deploy, monitor and troubleshoot a complete AWS web application from scratch.